Summarize

# Conclusion: A New Beginning?

Every company, regardless of its origins, is becoming a digital company. This transformation requires the delivery of APIs and services to be consumed by mobile applications, devices in the internet of things (IoT), or even autonomous vehicles and systems. The increasing criticality of these systems means that these online systems must be built for redundancy, fault tolerance, and high availability. At the same time, the requirements of business necessitate rapid agility to develop and roll out new software, iterate on existing applications, or experiment with new user interfaces and APIs. The confluence of these requirements has led to an order of magnitude increase in the number of distributed systems that need to be built.

The task of building these systems is still far too difficult. The overall cost of developing, updating, and maintaining such a system is far too high. Likewise, the set of people with the capabilities and skills to build such applications is far too small to address the growing need.

Historically, when these situations presented themselves in software development and technology, new abstraction layers and patterns of software development emerged to make building software faster, easier, and more reliable. This first occurred with the development of the first compilers and programming languages. Later, the development of object-oriented programming languages and managed code occurred. Likewise, at each of these moments, these technical developments crystallized the distillation of the knowledge and practices of experts into a series of algorithms and patterns that could be applied by a much wider group of practitioners. Technological advancement combined with the establishment of patterns democratized the process of developing software and expanded the set of developers who could build applications on the new platform. This in turn led to the development of more applications and application diversity, which in turn expanded the market for these developers' skills.

Again, we find ourselves at a moment of technological transformation. The need for distributed systems far exceeds our ability to deliver them. Fortunately, the development of technology has produced another set of tools to further expand the pool of develop-

ers capable of building these distributed systems. The recent development of containers and container orchestration has brought tools that enable rapid, easier development of distributed systems. With luck, these tools, when combined with the patterns and practices described in this book, can enhance and improve the distributed systems built by current developers, and more importantly, develop a whole new expanded group of developers capable of building these systems.

Patterns like sidecars, ambassadors, sharded services, FaaS, work queues, and more can form the foundation on which modern distributed systems are built. Distributed system developers should no longer be building their systems from scratch as individuals but rather collaborating together on reusable, shared implementations of canonical patterns that form the basis of all of the systems we collectively deploy. This will enable us to meet the demands of today's reliable, scalable APIs and services and empower a new set of applications and services for the future.

Previous chapter
< 16. Common Failure Patterns

Next chapter
Index >